# Rockchip Linux SDK 编译 ROS2 说明

文件标识：RK-SM-YF-912

发布版本：V1.1.2

日期：2026-04-11

文件密级：□绝密 □秘密 □内部资料 ■公开

**免责声明**

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

**商标声明**

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

**版权所有 © 2026 瑞芯微电子股份有限公司**

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

**前言**

**概述**

本文主要介绍如何在最新 Rockchip Linux SDK（基于 Buildroot）上编译 ROS2。与 ROS2 官方支持的 Debian 等系统不同，我们需要从源码逐一编译各个 ROS2 软件包。

Rockchip Linux SDK 早期版本曾集成 ROS1 编译支持，但已过时且无法继续维护。如需使用，请查阅早期（2021 年之前）的 SDK。

由于 ROS2 迭代速度快且对 Python 版本有强依赖，不同时期发布的 Rockchip Linux SDK 仅支持当时主流的 ROS2 版本。

当前 ROS2 的 LTS 版本主要为 Humble 和 Jazzy。早期的 Foxy、Iron 等版本已停止维护，如需使用请查阅 2023 年发布的 SDK。

Rockchip Linux SDK 源码仅提供 ROS2 所需的依赖包，ROS2 本身在特定的 Docker 环境中编译。这种设计将 ROS2 编译与 Rockchip Linux SDK 最大程度解耦，便于 ROS2 版本的更新维护。

ROS2 文档 [1][2] 显示，官方提供了 Docker 镜像用于交叉编译 arm/aarch64 架构（如 Dockerfile_Ubuntu_arm64_prebuilt[2]）。但目标系统仅支持 Ubuntu 18.04 （Bionic），与 Rockchip Linux SDK 的 rootfs 存在较大差异。借鉴该思路，我们将 ROS2 编译总结为三个步骤：

- 首先完成 Rockchip Linux SDK 编译，其中包含 ROS2 所需的依赖包（如 Python3、bullet、opencv、eigen 等）
- 进入 Docker 环境，利用 Ubuntu 系统及 Rockchip Linux SDK 的交叉编译工具链、Sysroot 交叉编译 ROS2
- 退出 Docker，重新打包 Rockchip Linux SDK rootfs

**产品版本**

| 芯片名称 | Rockchip Linux SDK 中Buildroot版本要求 |
| --- | --- |
| RK356x，RK3572，RK3576，RK3588，RV1126B等arm64位芯片 | tag:linux-6.12-stan-rka0 |

**读者对象**

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

**修订记录**

| 版本号 | 作者 | 修改日期 | 修改说明 |
| --- | --- | --- | --- |
| V1.0.0 | zhengsq | 2021-09-09 | 初始版本 |
| V1.1.0 | zhengsq | 2023-09-21 | 更新匹配当前新的Rockchip Linux SDK，<br>增加ROS2 humble及iron支持，<br>修改编译Docker环境，调整文档结构 |
| V1.1.1 | zhengsq | 2024-01-11 | 部分语句修正 |
| V1.1.2 | zhengsq | 2026-04-11 | 在最新SDK上支持ROS2 Jazzy/humble，并标记早期版本过时 |

**目录**


Rockchip Linux SDK 编译 ROS2 说明

1. ROS2的具体版本
2. 编译环境
3. 编译ROS2的依赖包
4. Host/Docker/Source准备
    4.1 下载ROS2 源码
    4.2 应用补丁
    4.3 复制脚本和配置
    4.4 构建 Docker 镜像
5. 进入Docker image编译ROS2
    5.1 单独编译某个ROS2 package 及应用程序
    5.2 打包rootfs并运行ROS2
6. 常见问题
    6.1 交叉编译Humble需要手动处理TRY_RUN
    6.2 需要UI显示的包不支持
7. 参考索引

# 1. ROS2的具体版本

支持以下2个LTS版本：

| ROS2 发行版本 | 详细版本号 | 链接 |
| --- | --- | --- |
| jazzy | ROS 2 Jazzy Jalisco - Patch Release 7 | https://github.com/ros2/ros2/releases/tag/release-jazzy-20260128 |
| humble | ROS 2 Humble Hawksbill - Patch Release 13 | https://github.com/ros2/ros2/releases/tag/release-humble-20250721 |

- 注： 其它未列出的ROS2如Foxy等版本，以及ROS1各版本，还未尝试过编译验证

# 2. 编译环境

建议编译Host：

- Ubuntu 20.04及其以上
- 参考Rockchip Linux SDK 编译的其它要求，在同一台机器上编译ROS2
- 网络需要能访问GitHub，大量源码包需要从GitHub下载

Rockchip Linux SDK 提供了统一的 ROS2 交叉编译脚本和补丁，位于 SDK 的 `docs/Patches/ROS2/` 目录下，目录结构如下：

```
ROS2/
├── build_ros2.sh           # 统一编译脚本（支持 humble/jazzy 参数）
├── rosdep.Dockerfile        # 统一 Dockerfile（支持 --build-arg ROS_DISTRO）
├── cross-compile/
│   ├── cross-compile.mixin  # CMake 交叉编译配置（两个版本共用）
│   └── index.yaml           # colcon mixin 索引文件
├── patches/
│   ├── humble/              # ROS2 Humble 补丁
│   │   ├── ament_cmake/
│   │   │   └── 0001-add-CMAKE_SYSTEM_PROCESSOR-support-for-cross-compila.patch
│   │   └── mimick_vendor/
│   │       └── 0001-Set-cross-compiling-CMAKE_SYSTEM_NAME-and-CMAKE_SYST.patch
│   └── jazzy/               # ROS2 Jazzy 补丁
│       └── ament_cmake/
│           └── 0001-add-CMAKE_SYSTEM_PROCESSOR-support-for-cross-compila.patch
```

支持的 ROS2 版本及补丁情况：

| 版本 | 发布时间 | 补丁数量 | 说明 |
| --- | --- | --- | --- |
| Humble | 2022-05 (LTS) | 2 个 | 需要 ament_cmake + mimick_vendor 补丁 |
| Jazzy | 2024-05 (LTS) | 1 个 | 仅需要 ament_cmake 补丁 |

在Rockchip Linux SDK的Buildroot目录中，检查是否存在相应的ros2 dependency config文件，

```
user@host:~/linux-sdk/buildroot$ ls -1 configs/rockchip/ros2/
configs/rockchip/ros2/jazzy_dep.config
configs/rockchip/ros2/humble_dep.config
```

- 如不存在该文件，请更新到新的SDK

# 3. 编译ROS2的依赖包

Rockchip Linux SDK 中的Buildroot工程里，configs/rockchip/ros2/目录下的配置文件提供了编译、运行ROS2所需要的依赖包，需要添加并编译到rootfs。例如，将对应版本的config（humble_dep.config或jazzy_dep.config）添加到rockchip_rk3572_defconfig中：

```
user@host:~/linux-sdk/buildroot$ git diff configs/rockchip_rk3572_defconfig
diff --git a/configs/rockchip_rk3572_defconfig b/configs/rockchip_rk3572_defconfig
index 8dc0afcd46..7072049b5c 100644
--- a/configs/rockchip_rk3572_defconfig
+++ b/configs/rockchip_rk3572_defconfig
@@ -22,3 +22,4 @@
 #include "network/chromium.config"
 #include "ai/npu2.config"
 #include "gui/weston.config"
+#include "ros2/jazzy_dep.config"
```

完整编译rootfs后，进入下一步。

# 4. Host/Docker/Source准备

Ubuntu PC机上安装docker程序：

```
sudo apt install docker.io
sudo usermod -aG docker $USER
newgrp docker  # 登录到docker用户组

sudo apt install python3-vcstool # 安装vcs-import工具
```

安装 qemu-user-binfmt（注册 binfmt 规则）

```
sudo apt install -y qemu-user-binfmt

# 验证安装（应看到 qemu-aarch64 等注册项）
ls /proc/sys/fs/binfmt_misc/qemu-*
/proc/sys/fs/binfmt_misc/qemu-aarch64
/proc/sys/fs/binfmt_misc/qemu-arm
...
```

**binfmt 作用说明**：

- binfmt_misc 是 Linux 内核功能，允许自动识别非本地架构的 ELF 二进制文件
- 注册后，执行 aarch64 二进制时内核会自动调用 `/usr/bin/qemu-aarch64-static`
- **注意**：binfmt 只负责启动 QEMU，还需要配置 `QEMU_LD_PREFIX` 告诉 QEMU 去哪里找目标系统的库

## 4.1 下载ROS2 源码

```
# 创建工作目录
mkdir -p ~/ros2 && cd ~/ros2

# 下载并解压 ROS2 发布包
# Humble:
wget https://github.com/ros2/ros2/archive/refs/tags/release-humble-20250721.tar.gz
tar zxf release-humble-20250721.tar.gz
# Jazzy:
wget https://github.com/ros2/ros2/archive/refs/tags/release-jazzy-20260128.tar.gz
tar zxf release-jazzy-20260128.tar.gz

# 使用 vcs-import 导入源码
mkdir src
# Humble 版本:
vcs-import -w 10 --retry 10 --skip-existing --recursive src < ros2-release-humble-
20250721/ros2.repos
# Jazzy 版本:
vcs-import -w 10 --retry 10 --skip-existing --recursive src < ros2-release-jazzy-
20260128/ros2.repos
```

- docker build时，会拷贝src源码，src目录需要与rosdep.Dockerfile在同一目录下

## 4.2 应用补丁

使用 `git am` 命令添加相应的补丁。补丁位于 SDK 的 `docs/Patches/ROS2/patches/` 目录下，按版本分目录存放：

```
SDK_PATCHES=~/rk/linux-sdk/docs/Patches/ROS2/patches

# Humble 对应的补丁（2个）
cp $SDK_PATCHES/humble/ament_cmake/*.patch src/ament/ament_cmake/
cp $SDK_PATCHES/humble/mimick_vendor/*.patch src/ros2/mimick_vendor/
cd src/ament/ament_cmake && git am *.patch && cd -
cd src/ros2/mimick_vendor && git am *.patch && cd -

# Jazzy 对应的补丁（1个）
cp $SDK_PATCHES/jazzy/ament_cmake/*.patch src/ament/ament_cmake/
cd src/ament/ament_cmake && git am *.patch && cd -
```

**补丁说明：**

| 补丁 | 适用版本 | 目的 | 应用位置 |
| --- | --- | --- | --- |
| ament_cmake | humble + jazzy | 添加 `CMAKE_SYSTEM_PROCESSOR` 交叉编译支持，使第三方库（如 googletest）能正确识别目标架构 | `src/ament/ament_cmake/` |
| mimick_vendor | 仅 humble | 设置交叉编译的 `CMAKE_SYSTEM_NAME` 和 `CMAKE_SYSTEM_PROCESSOR`，防止 mimick 尝试编译 HOST 架构的测试程序 | `src/ros2/mimick_vendor/` |

- 注：Jazzy 版本不需要 mimick_vendor 补丁，因为上游已修复相关问题

## 4.3 复制脚本和配置

将 SDK 提供的统一编译脚本、Dockerfile 和交叉编译配置复制到工作目录：

```
SDK_ROS2=~/rk/linux-sdk/docs/Patches/ROS2

cp $SDK_ROS2/build_ros2.sh .
cp $SDK_ROS2/rosdep.Dockerfile .
cp -r $SDK_ROS2/cross-compile .
chmod +x build_ros2.sh
```

## 4.4 构建 Docker 镜像

使用 Rockchip Linux SDK 提供的统一 `rosdep.Dockerfile` 创建 Docker 镜像，通过 `--build-arg ROS_DISTRO` 参数指定 ROS2 版本：

```
# Humble:
docker build --build-arg ROS_DISTRO=humble \
    --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
    -t humble-ros2-builder -f rosdep.Dockerfile ./

# Jazzy:
docker build --build-arg ROS_DISTRO=jazzy \
    --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
    -t jazzy-ros2-builder -f rosdep.Dockerfile ./
```

- Docker 中使用了与 host 的 user/group，保持 host 与 docker 对文件的访问权限一致
- 密码默认是： rockchip

# 5. 进入Docker image编译ROS2

将编译好的Rockchip Linux SDK 绑定到Docker环境中，执行编译：

```
cd ~/ros2

# 定义变量（根据实际情况修改）
export SDK_PATH=~/rk/linux-sdk
export BOARD_CONFIG=rockchip_rk3572_evb1_robot
export OUTPUT_DIR=${SDK_PATH}/buildroot/output/${BOARD_CONFIG}
export ROS_DISTRO=jazzy  # 或 humble
```

**使用当前用户权限运行（推荐，避免权限问题）**

```
# 进入 Docker image，保持 Host 用户权限
# 使用 --privileged 标志，让容器能访问主机的 binfmt
docker run -it --rm --privileged \
  -u $(id -u):$(id -g) \
  -v /etc/passwd:/etc/passwd:ro \
  -v /etc/group:/etc/group:ro \
  --mount type=bind,source=${OUTPUT_DIR},target=/buildroot \
  ${ROS_DISTRO}-ros2-builder:latest
```

**编译执行：**

```
user@host:~$ ls
build_ros2.sh  src

# 执行编译（支持 ROS_DISTRO 参数或环境变量）
user@host:~$ ./build_ros2.sh jazzy
# 或
user@host:~$ ROS_DISTRO=jazzy ./build_ros2.sh
....
Summary: 328 packages finished [1h 2min 16s]
....
```

**参数说明：**

- `source=`：需要修改成相应的 Rockchip Linux SDK 编译的 output 目录的**绝对路径**，该路径下应包含 target、host、staging、build 等目录
- 编译生成的目标文件位于 Rockchip Linux SDK 对应的 Image 目录：`${OUTPUT_DIR}/target/opt/ros`
- 编译中间过程存放在 `${OUTPUT_DIR}/build/ros` 目录

**编译时间参考：**

- 完整编译：约 1-2 小时（取决于 Host 性能，参考配置：16 核 CPU，32GB 内存，SSD）
- 增量编译：约 5-10 分钟（仅修改少量 package）

如build_ros2.sh未提示错误即成功编译。其中，还有部分包在Buildroot SDK环境中，无法编译、执行的，比如：

- rviz：依赖于X11/desktop。如果你需要这个功能，直接使用Ubuntu arm镜像，而不是Buildroot
- turtlesim：依赖于UI显示
- 如果想要取消某个包的编译，在src对应的路径下，创建一个COLCON_IGNORE即可。比如`touch src/ros/ros_tutorials/turtlesim/COLCON_IGNORE`

## 5.1 单独编译某个ROS2 package 及应用程序

参考`build_ros2.sh`脚本，使用colcon build的参数`--packages-select <package_name>`可单独编译包。

## 5.2 打包rootfs并运行ROS2

在上述ROS2完整编译结束后，进入到Rockchip Linux SDK，重新打包rootfs即可。ROS2安装在/opt/ros目录下。

```
cd ~/rk/linux-sdk
./build.sh rootfs   # 重新打包rootfs.img
```

烧录rootfs.img后，进入板端，执行Hello World Demo：

```
cd /opt/ros/
export COLCON_CURRENT_PREFIX=/opt/ros
export ROS_HOME=/userdata/
source ./local_setup.sh
ros2 pkg list
ros2 pkg executables
```

```
ros2 run demo_nodes_cpp listener &
ros2 run demo_nodes_cpp talker
[INFO] [1501839280.834017748] [talker]: Publishing: 'Hello World: 1'
[INFO] [1501839280.839280957] [listener]: I heard: [Hello World: 1]
[INFO] [1501839281.831636015] [talker]: Publishing: 'Hello World: 2'
[INFO] [1501839281.835092640] [listener]: I heard: [Hello World: 2]
[INFO] [1501839282.831618532] [talker]: Publishing: 'Hello World: 3'
[INFO] [1501839282.835336782] [listener]: I heard: [Hello World: 3]

ros2 run demo_nodes_py listener &
ros2 run demo_nodes_py talker
```

# 6. 常见问题

## 6.1 交叉编译Humble需要手动处理TRY_RUN

fastrtps TRY_RUN提示：

```
--- stderr: fastrtps
CMake Error: TRY_RUN() invoked in cross-compiling mode, please set the following cache
variables appropriately:
   SM_RUN_RESULT (advanced)
   SM_RUN_RESULT__TRYRUN_OUTPUT (advanced)
For details see /buildroot/build/ros/fastrtps/TryRunResults.cmake
```

- 需要按照说明，将应用程序放到板端执行，并按说明填写结果。例如：

```
user@host:~$ /tmp/cmTC_4f573-SM_RUN_RESULT
PTHREAD_RWLOCK_PREFER_READER_NP

# 根据上述执行结果，在docker中填入结果：
vim /buildroot/build/ros/fastrtps/TryRunResults.cmake
  ....
  set( SM_RUN_RESULT
       "0"
       CACHE STRING "PTHREAD_RWLOCK_PREFER_READER_NP" FORCE)

  set( SM_RUN_RESULT__TRYRUN_OUTPUT
       "0"
       CACHE STRING "PTHREAD_RWLOCK_PREFER_READER_NP" FORCE)
```

## 6.2 需要UI显示的包不支持

Rockchip Linux SDK不支持QT，因此如下一些依赖QT的包无法编译、运行：

- rviz相关包
- ros-visualization相关包，如rqt、python_qt_binding、qt_gui_core
- turtlesim

# 7. 参考索引


1. https://docs.ros.org/en/jazzy/Installation/Alternatives/Latest-Development-Setup.html
2. https://github.com/ros-tooling/cross_compile.git